THIS

DIRECTIVE "USE STRICT";

```
x = 3.14;
console.log(x); // 3.14
```

```
myFunction();

function myFunction() {
  y = 3.14;
  console.log(y); // 3.14
}
```

DIRECTIVE "USE STRICT";

Defines that JavaScript code should be executed in "strict mode".

With strict mode, you can not, for example, use undeclared variables.

```
'use strict';
x = 3.14; // ReferenceError: x is not defined
console.log(x);
```

DIRECTIVE "USE STRICT";

Defines that JavaScript code should be executed in "strict mode".

With strict mode, you can not, for example, use undeclared variables.

```
x = 3.14; // This will not cause an error.
myFunction();

function myFunction() {
  'use strict';
  y = 3.14; // ReferenceError: y is not defined
  console.log(y);
}
```

WHY STRICT MODE?

Strict mode makes it easier to write "secure" JavaScript.

In normal JavaScript, a developer will not receive any error feedback assigning values to non-writable properties.

In strict mode, any assignment to a non-writable property, a getter-only property, a non-existing property, a non-existing variable, or a non-existing object, will throw an error.

NOT ALLOWED IN STRICT MODE

Using a variable, without declaring it, is not allowed:

```
'use strict';
x = 3.14;
```

NOT ALLOWED IN STRICT MODE

Duplicating a parameter name is not allowed:

```
'use strict';
function x(p1, p1) {}
```

NOT ALLOWED IN STRICT MODE

Setting properties on primitive values is not allowed:

```
'use strict';

false.true = ''; // TypeError
(14).sailing = 'home'; // TypeError
'with'.you = 'far away'; // TypeError
```

NOT ALLOWED IN STRICT MODE

Full list is here https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Strict_mode

ARROW FUNCTION

ARROW FUNCTION

An arrow function expression () => {} is a compact alternative to a normal function expression with keyword function() {}

Normal function

```
function () {}
```

Arrow function

```
() => {};
```

ARROW FUNCTION SYNTAX

```
() => expression

param => expression

(param) => expression

(param1, paramN) => expression

() => {
  statements
}

param => {
  statements
}
```

ARROW FUNCTION EXAMPLES

```
const sum = (a, b) => a + b;
sum(5, 6); // 11
```

```
const arr = [1, 2, 3];
const mappedArr = arr.map((item, index) => {
  if (index % 2 === 0) {
    return item ** 2;
  } else {
    return item;
  }
});
console.log(mappedArr); // [1, 2, 9]
```

KEYWORD NEW

OBJECT CREATION USING KEYWORD NEW

Problem: what if there are 100 persons? Or 20-30 object properties like name, age, etc?

```
const ivan = { name: 'Ivan' };
const anton = { name: 'Anton' };
const maria = { name: 'Maria' };
//...
```

OBJECT CREATION USING KEYWORD NEW

Rough solution idea

```
function Person(name) {
  return {
    name: name,
  };
}

const ivan = Person('Ivan');
const anton = Person('Anton');
const maria = Person('Maria');
//...
```

OBJECT CREATION USING KEYWORD NEW

Better solution idea

```
function Person(name) {
  this.name = name; // keyword this referenced to the object you're going to c
}

const ivan = new Person('Ivan'); // keyword new is used to trigger object crea
const anton = new Person('Anton');
const maria = new Person('Maria');
//...
```

OBJECT CREATION USING KEYWORD NEW

Better solution idea

```
function Person(name, age) {
  this.name = name; // keyword this referenced to the object you're going to c
  this.age = age;
}

const persons = [
  new Person('Ivan', 24), // keyword new is used to trigger object creation
  new Person('Anton', 18),
  new Person('Maria', 20),
];
//...
```

KEYWORD THIS

KEYWORD THIS: ONE MORE USE CASE

```
const person = {
  name: 'Ivan',
};

function introduce() {
  console.log(`Hi, my name is ${person.name}`);
}

introduce();
```

```
const person = {
  name: 'Ivan',
  introduce: function () {
    console.log(`Hi, my name is ${person.name}`);
  },
};

person.introduce();
```

```
const person = {
  name: 'Ivan',
  introduce: function () {
    console.log(`Hi, my name is ${this.name}`);
  },
};

person.introduce();
```

KEYWORD THIS

this is a reference to any value

```
this;
```

```
function some() {
  return this;
}
```

```
function Person(name) {
  this.name = name;
}
const person = new Person('Ivan');
```

KEYWORD THIS

this could be perceived as a JavaScript expression, value of which is defined based on the execution context within a function runs

NORMAL FUNCTION, DOT NOTATION

Normal function *is a function that is different to an arrow function, meaning it's a function without symbols =>*

Dot notation *is a syntax when two JavaScript identifiers splitted from each other: person.name, person.introduce()*

THIS - HOW TO UNDERSTAND WHAT VALUE IT WILL BE REFERENCED TO?

See how a function is being called

THIS

When a function is being called, a new environment is being created for the function. And then the function is being executed.

A value of a keyword this depends on how a function is being called.

GLOBAL ENVIRONMENT

Host has a right to define this in global environment

Host: Browser

```
'use strict';
console.log(this); // Window
```

Host: Node.js 20.x

```
'use strict';
console.log(this); // {}
```

Host: Node.js 20.x

```
'use strict';
console.log(globalThis); // Object [global] { setImmediate: [Function: setImmediate], fet
```

On the next slides our Host will be a Browser

FIRST QUESTION FOR THIS - ARE WE INSIDE A FUNCTION? IF NO,

this is being set by Host in Global environment

```
'use strict';
console.log(this); // Window, we are outside of function
```

FIRST QUESTION FOR THIS - ARE WE INSIDE A FUNCTION? IF YES,

SECOND QUESTION FOR THIS - IS IT A NORMAL FUNCTION? IF YES,

this is being set by the way how the function was being called

```
'use strict';
function print() {
  console.log(this); // undefined because we are in "use strict" mode
}

print(); // call a normal function
```

```
function print() {
  console.log(this); // Window because we are not in "use strict" mode
}

print(); // call a normal function
```

FIRST QUESTION FOR THIS - ARE WE INSIDE A FUNCTION? IF YES,

SECOND QUESTION FOR THIS - IS IT A NORMAL FUNCTION? IF NO,

this for arrow function keeps the parent function environment

See examples on the slides further

THIRD QUESTION FOR THIS - ARE THERE CALL, APPLY, BIND

If yes, then this equals the value from call/apply/bind (first argument)

If no, then asking the next question

```
'use strict';
function print() {
  console.log(this);
}

print(); // this is undefined
print.call(null); // this is null
print.apply({ name: 'Ivan' }); // this is { name: 'Ivan' }
print.bind(54)(); // this is 54
```

FOURTH QUESTION FOR THIS - IS THERE KEYWORD NEW

If yes, then this equals {}

If no, then asking the next question

```
'use strict';

function Person() {
  console.log(this);
}

const person = new Person(); // this is {}
```

FIFTH QUESTION FOR THIS - DOT-NOTATION

If a function is called using dot-notation, then this equals object from the left side from dot

If no, then by default this is undefined ("use strict"; mode) or what Host defines

```
'use strict';
function print() {
  console.log(this);
}

var ivan = { name: 'Ivan' };
ivan.print = print;
ivan.print(); // this is { name: 'Ivan', print: f }
```

```
'use strict';
const ivan = {
  name: 'Ivan',
  print: function () {
    console.log(this);
  },
};
ivan.print(); // this is { name: 'Ivan', print: f }
```

FIFTH QUESTION FOR THIS - DOT-NOTATION

If a function is called using dot-notation, then this equals object from the left side from dot

If no, then by default this is undefined ("use strict"; mode) or what Host defines

```
'use strict';
function Person(name) {
  this.name = name;
  this.print = function () {
    console.log(this);
  };
}

const ivan = new Person('Ivan');
ivan.print(); // this is Person { name: 'Ivan', print: f }
```

FIFTH QUESTION FOR THIS - DOT-NOTATION

If a function is called using dot-notation, then this equals object from the left side from dot

If no, then by default this is undefined ("use strict"; mode) or what Host defines

```
'use strict';
function Person(name) {
  this.name = name;
  this.print = () => {
    console.log(this);
  };
}

const ivan = new Person('Ivan');
ivan.print(); // this is Person { name: 'Ivan', person: f }
```

FIFTH QUESTION FOR THIS - DOT-NOTATION

If a function is called using dot-notation, then this equals object from the left side from dot

If no, then by default this is undefined ("use strict"; mode) or what Host defines

```
'use strict';
function Person(name) {
  return {
    name,
    print: function () {
      console.log(this);
    },
  };
}

const ivan = Person('Ivan');
ivan.print(); // this is { name: 'Ivan', print: f }
```

```
'use strict';
function Person(name) {
  return {
    name,
    print: () => {
      console.log(this);
    },
  };
}

const ivan = Person('Ivan');
ivan.print(); // this is undefined
```

FUNCTIONS CALL, APPLY, BIND

FUNCTION CALL

__Call__ is a function that helps you change the environment of the invoking function. It helps you replace the value of this inside a function with whatever value you want.

```
func.call(thisObj, args1, args2, ...)
```

- **func** is a function that needs to be invoked with a different this object
- **thisObj** is an object or a value that needs to be replaced with the this keyword present inside the function func
- **args1, args2** are arguments that are passed to the invoking function with the changed this object.

FUNCTION CALL

```
function greet(name) {
  console.log(`Hello, ${name}! My name is ${this.name}.`);
}

let person = {
  name: 'John',
};

greet.call(person, 'Alice'); // prints 'Hello, Alice! My name
```

FUNCTION CALL

```
const ivan = {
  name: 'Ivan',
  introduce: function () {
    console.log(this.name);
  },
};

const maria = { name: 'Maria' };

ivan.introduce(); // prints 'Ivan'
maria.introduce(); // TypeError: maria.introduce is not a func
ivan.introduce.call(maria); // prints 'Maria'
```

FUNCTION APPLY

Apply is a function that helps you change the environment of the invoking function. It helps you replace the value of this inside a function with whatever value you want.

```
func.apply(thisObj, [args1, args2, ...])
```

- **func** is a function that needs to be invoked with a different this object
- **thisObj** is an object or a value that needs to be replaced with the this keyword present inside the function func
- **[args1, args2, ...]** is an array of arguments that are passed to the invoking function with the changed this object.

FUNCTION APPLY

```
function greet(name, age) {
  console.log(
    `Hello, ${name}! My name is ${this.name}. I heard you are
  );
}

let person = {
  name: 'John',
};

greet.apply(person, ['Alice', 25]); // prints 'Hello, Alice! M
```

FUNCTION APPLY

```
const ivan = {
  name: 'Ivan',
  introduce: function () {
    console.log(this.name);
  },
};

const maria = { name: 'Maria' };

ivan.introduce(); // prints 'Ivan'
maria.introduce(); // TypeError: maria.introduce is not a func
ivan.introduce.apply(maria); // prints 'Maria'
```

FUNCTION BIND

Bind *is a function that helps you create another function that you can execute later with the environment of this that is provided.*

```
func.bind(thisObj, arg1, arg2, ..., argN);
```

- **func** is a function that needs to be invoked with a different this object
- **thisObj** is an object or a value that needs to be replaced with the this keyword present inside the function func
- **arg1, arg2…argN** are arguments that are passed to the invoking function with the changed this object.

FUNCTION BIND

```
function greet(name, age) {
  console.log(
    `Hello, ${name}! My name is ${this.name}. I heard you are
  );
}

let person = {
  name: 'John',
};

const greetLater = greet.bind(person, 'Alice', 25); // don't c
greetLater(); // prints 'Hello, Alice! My name is John. I hear

const greetLater2 = gree.bind(person);
greetLater2('Ivan', 30); // prints 'Hello, Ivan! My name is Jo
```

USED MATERIALS

- https://www.youtube.com/watch?v=fQ7_GT8_zeM
- https://docs.google.com/presentation/d/1LDJMSHmTkrYYdXQCinUIAQ9BMV2jIrctHhMfGSJI
  usp=sharing
- https://drive.google.com/drive/folders/1n8pXffdvd9DeNpeXGR8rBbFUM8Hxa_ms
- https://www.w3schools.com/js/js_strict.asp
- https://www.w3schools.com/js/js_this.asp
- https://www.w3schools.com/js/js_arrow_function.asp
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/new